Lecture 3 revision sheet

Linear algebra 2: eigenvalues, diagonalisation, the SVD and systems of ODEs

This sheet lists the definitions, formulas and results from Lecture 3, without derivations or discussion. These are in the full notes (`linear-algebra-2.ipynb`, homework and supplementary notebooks); the grey pointers give the section. → Ln marks material that later lectures build on. [*] marks material that is optional in the notes, including everything that is only in the supplementary notebook.

Key messages

1. A square matrix maps the unit circle (sphere) to an ellipse (ellipsoid). **Eigenvectors** are the directions it only scales: $A\boldsymbol{v} = \lambda\boldsymbol{v}$ (a negative λ reverses the direction). They are the transformation's "natural axes".
2. Eigenvalues are the roots of the **characteristic polynomial** $\det(A - \lambda I) = 0$. A real matrix can have complex (conjugate-pair) and repeated eigenvalues.
3. If A has n linearly independent eigenvectors it can be **diagonalised**, $A = P\Lambda P^{-1}$: change to eigen-coordinates, scale, change back. A real symmetric matrix always can, with an orthogonal P.
4. The **SVD** $A = U\Sigma V^T$ exists for *every* (also non-square) matrix: rotate/reflect, scale (and change dimension), rotate/reflect.
5. Diagonalisation **decouples** a linear ODE system $\dot{\boldsymbol{x}} = A\boldsymbol{x}$ into scalar problems $\dot{z}_i = \lambda_i z_i$, so the eigenvalues decide the stability of the solution, and of a numerical method applied to it.

1 Linear transformations and bases

Full notes: *Geometric Interpretation of Linear Transformations (continued)*

- **Unit circle → ellipse:** $\boldsymbol{x} \mapsto A\boldsymbol{x}$ maps the unit circle $\|\boldsymbol{x}\|_2 = 1$ to an ellipse. The images of the basis vectors $\boldsymbol{i}, \boldsymbol{j}$ are the columns of A (L2). Most vectors change direction and length; eigenvectors only change length. For a *symmetric* A the eigenvectors lie along the ellipse's major and minor axes.
- **Transforming a vector** (the point moves): $\boldsymbol{p}' = A\boldsymbol{p}$, both in the standard basis.
- **Changing basis** (the point stays put): if the columns of A_b are the new basis vectors, the coordinates of $\boldsymbol{p}$ in the new basis satisfy $A_b\boldsymbol{p}' = \boldsymbol{p}$, i.e. $\boldsymbol{p}' = A_b^{-1}\boldsymbol{p}$.
- **A transformation in a new basis** [*]**:** the matrix that applies A to coordinates written in the basis A_b is the *similarity transform* $A' = A_b^{-1}AA_b$ (convert to standard coordinates, apply A, convert back). Choosing the eigenvectors as the basis makes A' diagonal (§3).

2 Eigenvalues and eigenvectors

Full notes: *Eigenvalues and eigenvectors; The determinant again...* → **used in L4, L6–L8**

For a square $n \times n$ matrix A, a scalar λ and a *non-zero* vector $\boldsymbol{v}$ with

$$A\boldsymbol{v} = \lambda\boldsymbol{v} \iff (A - \lambda I)\boldsymbol{v} = \boldsymbol{0}$$

are an eigenvalue and eigenvector. A non-zero solution needs $A - \lambda I$ to be singular:

$$\det(A - \lambda I) = 0 \qquad \text{(the characteristic polynomial, degree } n\text{, so } n \text{ roots counted with multiplicity).}$$

- **Recipe:** find the roots λ_i, then for each one solve $(A - \lambda_i I)\boldsymbol{v} = \boldsymbol{0}$. Eigenvectors are only defined up to a scalar multiple; it is common to *normalise* them, $\|\boldsymbol{v}\|_2 = \sqrt{\boldsymbol{v}^T\boldsymbol{v}} = 1$.
- **Worked example:** $A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$: $\det(A - \lambda I) = \lambda^2 - 6\lambda + 8 = (\lambda - 4)(\lambda - 2)$, so $\lambda_1 = 4$, $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}}(1, 1)^T$ and $\lambda_2 = 2$, $\boldsymbol{v}_2 = \frac{1}{\sqrt{2}}(-1, 1)^T$. These are *orthonormal*: unit length and $\boldsymbol{v}_1 \cdot \boldsymbol{v}_2 = 0$.
- Real matrices can have complex eigenvalues; these come in complex-conjugate pairs. SciPy's `sl.eig` always returns complex numbers (take `np.real` when you know they are real).
- **Linear independence:** $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$ are l.i. if $\sum_i \alpha_i \boldsymbol{v}_i = \boldsymbol{0}$ has only the solution $\alpha_i = 0$ for all i; equivalently none is a linear combination of the others, and n of them span n-dimensional space.
- **Projection** onto a unit vector $\boldsymbol{v}$ uses the projection matrix $\boldsymbol{v}\boldsymbol{v}^T$ (L2 homework). Projecting onto eigenvectors or singular vectors is the basis of PCA (§4).

Repeated eigenvalues: algebraic and geometric multiplicity

Algebraic multiplicity	How many times λ is a root of the characteristic polynomial.
Geometric multiplicity	The dimension of its *eigenspace*, i.e. of the null space of $A - \lambda I$ (the number of l.i. eigenvectors for λ). Always $\leq$ the algebraic multiplicity.
$I_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$	$\lambda = 1$ twice; two orthogonal eigenvectors: algebraic = geometric = 2.
$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$	$\lambda = 1$ twice; only one eigenvector direction $(1, 0)^T$: algebraic 2, geometric 1.

Useful results (stated without proof)

1. A real **symmetric** matrix has *real* eigenvalues.
2. Eigenvectors for *distinct* eigenvalues are linearly independent.
3. For a real symmetric matrix, eigenvectors for distinct eigenvalues are *orthogonal*; it always has n mutually orthogonal eigenvectors (even with repeated eigenvalues, after choosing them appropriately, e.g. by Gram–Schmidt). Normalised, they form an orthonormal basis and $P^T P = I$.

Symmetric positive-definite (SPD) matrices [*]

- A symmetric A is **positive definite** if $\boldsymbol{x}^T A \boldsymbol{x} > 0$ for all $\boldsymbol{x} \neq \boldsymbol{0}$ $\iff$ all its eigenvalues are strictly positive. (Semi-definite, negative-definite are defined similarly.)
- **Strictly diagonally dominant:** $|A_{ii}| > \sum_{j \neq i} |A_{ij}|$ for every row i. A symmetric, strictly diagonally dominant matrix with positive diagonal is SPD (a quick sufficient check, not a necessary one).

3 Matrix diagonalisation

Full notes: *Matrix diagonalisation; Orthogonal Matrices: Rotations and Reflections* → **used in L4, L6**

With $P = [\, \boldsymbol{v}_1 \; \boldsymbol{v}_2 \; \cdots \; \boldsymbol{v}_n \,]$ (eigenvectors as columns) and $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$, we always have $AP = P\Lambda$. If the eigenvectors are linearly independent, P is invertible and

$$A = P\Lambda P^{-1} \qquad \iff \qquad P^{-1} A P = \Lambda.$$

- **When is A diagonalisable?** n distinct eigenvalues is sufficient (not necessary). In general [*]: iff it has n l.i. eigenvectors $\iff$ algebraic = geometric multiplicity for every eigenvalue. Otherwise the Jordan normal form is the best available. Example: $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ is not diagonalisable.
- **Real symmetric A:** always diagonalisable by an *orthogonal* matrix, $A = Q\Lambda Q^T \iff \Lambda = Q^T A Q$.
- **Orthogonal matrix Q**: columns (and rows) are orthonormal, $Q^T Q = Q Q^T = I$. Hence $Q^{-1} = Q^T$, $\|Q\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2$ and $(Q\boldsymbol{x}) \cdot (Q\boldsymbol{y}) = \boldsymbol{x} \cdot \boldsymbol{y}$: lengths and angles are preserved, so Q is a rotation, a reflection, or a combination.
- **Geometric reading** of $A\boldsymbol{x} = P\left(\Lambda\left(P^{-1}\boldsymbol{x}\right)\right)$: P^{-1} changes to eigenvector coordinates, Λ stretches each coordinate by λ_i, P changes back. For symmetric A the first and last steps are rotations/reflections ($P^{-1} = P^T$); for non-symmetric A they are general changes of basis.
- **Powers** [*]**:** $A^N = P\Lambda^N P^{-1}$ with $\Lambda^N = \mathrm{diag}(\lambda_1^N, \ldots, \lambda_n^N)$, much cheaper than N matrix products. → **used in L4**
- **Fibonacci matrix:** $Q = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ has $\det(Q - \lambda I) = \lambda^2 - \lambda - 1$, so $\lambda_{1,2} = \frac{1 \pm \sqrt{5}}{2}$ with eigenvectors $(\lambda_{1,2}, 1)^T$. With Q^k this gives any Fibonacci number (L1 recurrence). → **used in L4**
- **Linear systems** [*]**:** if $A = P\Lambda P^{-1}$ then $\boldsymbol{x} = P\Lambda^{-1}P^{-1}\boldsymbol{b}$. Too expensive in practice (finding eigenvalues is hard for large matrices), but the idea of transforming to an easier problem underlies preconditioning.

4 Singular value decomposition (SVD)

Full notes: *Singular Value Decomposition (SVD)* → **used in L4, L8**

Every real $m \times n$ matrix can be factorised as

$$A = U\Sigma V^T$$

Factor	Size	Description
U	$m \times m$ orthogonal	columns: *left singular vectors*, eigenvectors of AA^T (orthonormal basis of $\mathbb{R}^m$)
Σ	$m \times n$ diagonal	*singular values* $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0$, $p = \min(m, n)$
V	$n \times n$ orthogonal	columns: *right singular vectors*, eigenvectors of A^TA (orthonormal basis of $\mathbb{R}^n$)

$\sigma_i = \sqrt{\text{eigenvalues of } A^TA}$ (equivalently of AA^T; the larger of the two has extra zero eigenvalues), so at most $\min(m, n)$ singular values are non-zero.

- **Geometric reading** of $A\boldsymbol{x} = U(\Sigma(V^T\boldsymbol{x}))$: V^T rotates/reflects (a change of basis to the right singular vectors); Σ scales each component by σ_i and pads with zeros or truncates if $m \neq n$; U rotates/reflects in the output space. The unit sphere is mapped to a (hyper)ellipse with semi-axes of length σ_i along the columns $\boldsymbol{u}_i$ of U.
- **Compact (reduced) SVD:** with r the number of non-zero singular values ($r = \operatorname{rank} A$), keep only those: $A = U_r\Sigma_r V_r^T$, with U_r ($m \times r$) and V_r ($n \times r$) having orthonormal columns ($U_r^TU_r = V_r^TV_r = I_r$; not "orthogonal", which needs a square matrix) and Σ_r ($r \times r$) diagonal.
- **Outer-product form:** $A = \sum_i \sigma_i \boldsymbol{u}_i\boldsymbol{v}_i^T$. Truncating after k terms gives a *low-rank approximation* (e.g. image compression).
- **SVD vs eigendecomposition:** for a symmetric positive semi-definite matrix they coincide ($U = V = P$, $\Sigma = \Lambda$, up to signs). For a general symmetric matrix $\sigma_i = |\lambda_i|$. For a non-symmetric matrix they are different factorisations; the SVD always exists, the eigendecomposition may not.
- **Signs:** singular vectors are only defined up to sign, so computing U and V separately from AA^T and A^TA can give an inconsistent pair; fix the signs so that $U\Sigma V^T = A$ (e.g. compare with `sl.svd`).
- **PCA:** project data onto the first few right singular vectors (largest σ_i), using the projection $\sum_i \boldsymbol{v}_i\boldsymbol{v}_i^T$ onto their span, to reduce dimension while keeping the directions of largest variance. → **used in L8**

5 Systems of linear ODEs [*]

Full notes: *supplementary: Using linear algebra to solve systems of ODEs* → **used in L6, L7**

For $\dot{\boldsymbol{x}} = A\boldsymbol{x}$ with $\boldsymbol{x}(0) = \boldsymbol{x}_0$:

$$\boldsymbol{x}(t) = \exp(tA)\,\boldsymbol{x}_0, \qquad \exp(tA) = I + tA + \tfrac{t^2}{2!}A^2 + \cdots = P\,\operatorname{diag}\left(e^{\lambda_1 t}, \ldots, e^{\lambda_n t}\right)P^{-1} \quad (\text{if } A = P\Lambda P^{-1}).$$

Equivalently, substitute $\boldsymbol{x} = P\boldsymbol{z}$: the system **decouples** into $\dot{z}_i = \lambda_i z_i$, so $z_i(t) = z_i(0)e^{\lambda_i t}$ and $\boldsymbol{x}(t) = \sum_i z_i(0)\, e^{\lambda_i t}\, \boldsymbol{v}_i$.

- **Example:** $A = \begin{pmatrix} -1 & 3 \\ 3 & -1 \end{pmatrix}$ has $\lambda_1 = 2$, $\boldsymbol{v}_1 = (1, 1)^T$ and $\lambda_2 = -4$, $\boldsymbol{v}_2 = (-1, 1)^T$, giving $\boldsymbol{x}(t) = \frac{1}{2}(C_1 + C_2)\,(1, 1)^T e^{2t} + \frac{1}{2}(C_2 - C_1)\,(-1, 1)^T e^{-4t}$ with $\boldsymbol{C} = \boldsymbol{x}(0)$. Only initial conditions with $C_1 + C_2 = 0$ decay to zero.
- **Stability of the continuous problem:** each mode behaves like $z = e^{\lambda t} = e^{\operatorname{Re}(\lambda)t}\big(\cos(\operatorname{Im}(\lambda)t) + i\sin(\operatorname{Im}(\lambda)t)\big)$. It decays to zero (stable) iff $\operatorname{Re}(\lambda) < 0$; it oscillates with constant amplitude if $\operatorname{Re}(\lambda) = 0$; it grows if $\operatorname{Re}(\lambda) > 0$. The imaginary part only gives bounded oscillation.

The mass–spring–damper (MSD) problem [*]

→ **used in L6**

- $m\ddot{x}+c\dot{x}+kx=0$, i.e. $\ddot{x}+2\zeta\omega_0\dot{x}+\omega_0^2 x=0$, with natural frequency $\omega_0=\sqrt{k/m}$ and damping ratio $\zeta=c/(2\sqrt{mk})$.
- **As a first-order system** ($v=\dot{x}$, $\boldsymbol{y}=(x,v)^T$): $\dot{\boldsymbol{y}}=A\boldsymbol{y}$ with $A=\begin{pmatrix}0 & 1\\ -\omega_0^2 & -2\zeta\omega_0\end{pmatrix}$. This is the standard way to hand a higher-order ODE to a first-order solver.
- **Eigenvalues:** $\lambda^2+2\zeta\omega_0\lambda+\omega_0^2=0 \Rightarrow \lambda_{1,2}=\omega_0\big(-\zeta\pm\sqrt{\zeta^2-1}\big)$.

Damping ratio	Eigenvalues	Behaviour
$\zeta=0$	purely imaginary, $\pm i\omega_0$	undamped (simple harmonic motion)
$0<\zeta<1$	complex pair, $\mathrm{Re}(\lambda)=-\zeta\omega_0<0$	under-damped (decaying oscillation)
$\zeta=1$	repeated, $-\omega_0$ (A not diagonalisable)	critically damped
$\zeta>1$	real, distinct, negative	over-damped

Exact under-damped solution ($0<\zeta<1$, $x(0)=x_0$, $\dot{x}(0)=v_0$), useful for verifying a numerical solver:

$$x(t)=2|C|\,e^{-\zeta\omega_0 t}\cos(\omega t+\phi),\qquad \omega=\omega_0\sqrt{1-\zeta^2},\qquad \tan\phi=\frac{-v_0-\zeta\omega_0 x_0}{\omega x_0},\qquad 2|C|=\frac{x_0}{\cos\phi}$$

(compute ϕ with `np.arctan2`, which picks the correct quadrant).

6 Forward Euler for linear systems [*]

Full notes: *supplementary: Discretisation with forward Euler; homework: stability of forward Euler…* → **used in L6, L7**

- Forward Euler (L1) on $\dot{\boldsymbol{y}}=A\boldsymbol{y}$: $\dfrac{\boldsymbol{y}_{n+1}-\boldsymbol{y}_n}{\Delta t}=A\boldsymbol{y}_n \iff \boldsymbol{y}_{n+1}=(I+A\Delta t)\,\boldsymbol{y}_n$, so $\boldsymbol{y}_n=(I+A\Delta t)^n\boldsymbol{y}_0$. Each step is one matrix–vector product.
- In eigen-coordinates each mode is multiplied by $1+\lambda_i\Delta t$ every step, so the numerical solution behaves well only if the method handles the scalar problem $y'=\lambda y$ well for every eigenvalue (which may be complex).
- **The numerical solution can blow up when the exact one decays.** Homework examples:

Problem	Parameters	$\lvert 1+\lambda\Delta t\rvert$	Forward Euler
$A=\begin{pmatrix}-1 & 3\\ 3 & -1\end{pmatrix}$, $\boldsymbol{x}_0=(1,-1)^T$ ($\lambda=-4$ mode)	$\Delta t=0.1$ / $\Delta t=1$	0.6 / 3	decays / blows up
MSD, $\omega_0=1$	$\Delta t=0.05$, $\zeta=0.05$	0.9987	stable
MSD, $\omega_0=1$	$\Delta t=0.2$, $\zeta=0.05$	1.0100	unstable
MSD, $\omega_0=1$	$\Delta t=0.2$, $\zeta=0.2$	0.9798	stable

Looking ahead (L6). The condition is $|1+\lambda\Delta t|<1$ for every eigenvalue λ: the forward Euler *amplification factor*, as in L1. In the first example $1+\lambda\Delta t=1-4\Delta t$, so it is stable iff $\Delta t<0.5$. For the under-damped MSD, $|1+\lambda\Delta t|^2=1-2\zeta\omega_0\Delta t+\omega_0^2\Delta t^2$, so forward Euler is stable iff $\Delta t<2\zeta/\omega_0$; with $\zeta=0$ it is unstable for every Δt. Stability again depends on the physics (ζ,ω_0) together with the numerics (Δt).

Better solvers: `scipy.integrate.solve_ivp` (method `'RK45'`: higher order, adaptive Δt) is used for the coupled two-mass MSD system and the double and triple pendulums, whose small changes in initial conditions grow rapidly (chaos). → **used in L7**

7 Python summary

`lam, P = sl.eig(A)`	eigenvalues (complex dtype) and normalised eigenvectors as *columns* of `P`
`sl.eigvals(A)`	eigenvalues only
`U, s, VT = sl.svd(A)`	SVD; `s` is a 1D array, so build an $m\times n$ `Sigma` to check `U @ Sigma @ VT`
`np.allclose(A, P @ np.diag(lam) @ sl.inv(P))`	check a factorisation (allow for round-off)
`np.linalg.matrix_power(A, N)`	A^N
`solve_ivp(f, [t0, tf], y0, method='RK45', t_eval=ts)`	accurate ODE-system solver; `f(t, y)` returns the RHS